

Module 10.2 - Reactive Forms: Validation and Controls

Learning Objectives:

- Implement synchronous validators in reactive forms
 - Use built-in validators (required, min, max, minLength, maxLength, pattern)
 - Create custom synchronous validators
 - Display validation error messages effectively
 - Implement cross-field validation
 - Add and remove validators dynamically
-

Table of Contents

1. [Built-in Validators](#)
 2. [Displaying Error Messages](#)
 3. [Multiple Validators](#)
 4. [Custom Validators](#)
 5. [Cross-Field Validation](#)
 6. [Dynamic Validation](#)
 7. [Practical Example: User Registration with Validation](#)
 8. [Practical Task](#)
 9. [Summary](#)
-

1. Built-in Validators

1.1 Required Validator

Component:

```
import { Component } from '@angular/core';
import { FormBuilder, FormGroup, Validators } from '@angular/forms';

@Component({
  selector: 'app-name-form',
  templateUrl: './name-form.component.html'
})
export class NameFormComponent {
  nameForm: FormGroup;

  constructor(private fb: FormBuilder) {
    this.nameForm = this.fb.group({
      name: ['', Validators.required]
    });
  }

  get name() {
    return this.nameForm.get('name');
  }

  onSubmit() {
    console.log('Name:', this.nameForm.value);
  }
}
```

Template:

```
<form [formGroup]="nameForm" (ngSubmit)="onSubmit()">
  <div>
    <label>Name:</label>
    <input type="text" formControlName="name">
  </div>

  <p>Valid: {{ name?.valid }}</p>
  <p>Invalid: {{ name?.invalid }}</p>

  <button type="submit" [disabled]="nameForm.invalid">Submit</button>
</form>
```

Output:

Initial state (empty field): - Valid: false - Invalid: true - Submit button disabled
User types "John": - Valid: true - Invalid: false - Submit button enabled

1.2 MinLength and MaxLength Validators

Component:

```
import { Component } from '@angular/core';
import { FormBuilder, FormGroup, Validators } from '@angular/forms';

@Component({
  selector: 'app-username-form',
  templateUrl: './username-form.component.html'
})
export class UsernameFormComponent {
  usernameForm: FormGroup;

  constructor(private fb: FormBuilder) {
    this.usernameForm = this.fb.group({
      username: ['', [
        Validators.required,
        Validators.minLength(3),
        Validators.maxLength(15)
      ]]
    });
  }

  get username() {
    return this.usernameForm.get('username');
  }
}
```

Template:

```
<form [formGroup]="usernameForm">
  <div>
    <label>Username:</label>
    <input type="text" formControlName="username">
  </div>
```

```
<p>Value Length: {{ username?.value?.length || 0 }}</p>
<p>Valid: {{ username?.valid }}</p>
</form>
```

Output:

User types "ab": - Value Length: 2 - Valid: false (less than minLength 3) User types
"abc": - Value Length: 3 - Valid: true (meets minLength) User types
"abcdefghijklmnop": - Value Length: 16 - Valid: false (exceeds maxLength 15)

1.3 Min and Max Validators (Numbers)

Component:

```
import { Component } from '@angular/core';
import { FormBuilder, FormGroup, Validators } from '@angular/forms';

@Component({
  selector: 'app-age-form',
  templateUrl: './age-form.component.html'
})
export class AgeFormComponent {
  ageForm: FormGroup;

  constructor(private fb: FormBuilder) {
    this.ageForm = this.fb.group({
      age: ['', [
        Validators.required,
        Validators.min(18),
        Validators.max(65)
      ]]
    });
  }

  get age() {
    return this.ageForm.get('age');
  }
}
```

Template:

```
<form [formGroup]="ageForm">
  <div>
    <label>Age:</label>
    <input type="number" formControlName="age">
  </div>

  <p>Value: {{ age?.value }}</p>
  <p>Valid: {{ age?.valid }}</p>
</form>
```

Output:

User enters 15: - Value: 15 - Valid: false (less than min 18) User enters 25: - Value: 25 - Valid: true (within range 18-65) User enters 70: - Value: 70 - Valid: false (exceeds max 65)

1.4 Pattern Validator (Regular Expressions)

Component:

```
import { Component } from '@angular/core';
import { FormBuilder, FormGroup, Validators } from '@angular/forms';

@Component({
  selector: 'app-phone-form',
  templateUrl: './phone-form.component.html'
})
export class PhoneFormComponent {
  phoneForm: FormGroup;

  constructor(private fb: FormBuilder) {
    this.phoneForm = this.fb.group({
      phone: ['', [
        Validators.required,
        Validators.pattern('^[0-9]{10}$') // Exactly 10 digits
      ]]
    });
  }

  get phone() {
```

```
    return this.phoneForm.get('phone');  
  }  
}
```

Template:

```
<form [formGroup]="phoneForm">  
  <div>  
    <label>Phone (10 digits):</label>  
    <input type="tel" formControlName="phone">  
  </div>  
  
  <p>Valid: {{ phone?.valid }}</p>  
</form>
```

Output:

User types "12345": - Valid: false (only 5 digits) User types "1234567890": - Valid: true (exactly 10 digits) User types "123abc7890": - Valid: false (contains non-numeric characters)

1.5 Email Validator

Component:

```
import { Component } from '@angular/core';  
import { FormBuilder, FormGroup, Validators } from '@angular/forms';  
  
@Component({  
  selector: 'app-email-form',  
  templateUrl: './email-form.component.html'  
})  
export class EmailFormComponent {  
  emailForm: FormGroup;  
  
  constructor(private fb: FormBuilder) {  
    this.emailForm = this.fb.group({  
      email: ['', [Validators.required, Validators.email]]  
    });  
  }  
}
```

```
get email() {  
  return this.emailForm.get('email');  
}  
}
```

Template:

```
<form [formGroup]="emailForm">  
  <div>  
    <label>Email:</label>  
    <input type="email" formControlName="email">  
  </div>  
  
  <p>Valid: {{ email?.valid }}</p>  
</form>
```

Output:

User types "john": - Valid: false (not a valid email format) User types "john@": - Valid: false (incomplete email) User types "john@example.com": - Valid: true (valid email format)

2. Displaying Error Messages

2.1 Basic Error Display

Component:

```
import { Component } from '@angular/core';  
import { FormBuilder, FormGroup, Validators } from '@angular/forms';  
  
@Component({  
  selector: 'app-login-form',  
  templateUrl: './login-form.component.html'  
})  
export class LoginFormComponent {
```

```
loginForm: FormGroup;

constructor(private fb: FormBuilder) {
  this.loginForm = this.fb.group({
    username: ['', [Validators.required, Validators.minLength(3)]],
    password: ['', [Validators.required, Validators.minLength(6)]]
  });
}

get username() {
  return this.loginForm.get('username');
}

get password() {
  return this.loginForm.get('password');
}
}
```

Template:

```
<form [formGroup]="loginForm">
  <div>
    <label>Username:</label>
    <input type="text" formControlName="username">

    <div *ngIf="username?.invalid && username?.touched" style="color: red;">
      <div *ngIf="username?.errors?.['required']">
        Username is required
      </div>
      <div *ngIf="username?.errors?.['minlength']">
        Username must be at least 3 characters
      </div>
    </div>
  </div>

  <div>
    <label>Password:</label>
    <input type="password" formControlName="password">

    <div *ngIf="password?.invalid && password?.touched" style="color: red;">
      <div *ngIf="password?.errors?.['required']">
        Password is required
      </div>
      <div *ngIf="password?.errors?.['minlength']">
        Password must be at least 6 characters
      </div>
    </div>
  </div>
</form>
```



```
</div>
</div>
</div>

<button type="submit" [disabled]="loginForm.invalid">Login</button>
</form>
```

Output:

User clicks on username field and leaves it empty: - Error appears: "Username is required" User types "ab" in username: - Error appears: "Username must be at least 3 characters" User types "abc": - Error disappears (valid) User clicks password field, types "12345": - Error appears: "Password must be at least 6 characters" User adds one more character "123456": - Error disappears (valid)

2.2 Error Messages with Details

Component:

```
import { Component } from '@angular/core';
import { FormBuilder, FormGroup, Validators } from '@angular/forms';

@Component({
  selector: 'app-bio-form',
  templateUrl: './bio-form.component.html'
})
export class BioFormComponent {
  bioForm: FormGroup;

  constructor(private fb: FormBuilder) {
    this.bioForm = this.fb.group({
      bio: ['', [
        Validators.required,
        Validators.minLength(10),
        Validators.maxLength(200)
      ]]
    });
  }

  get bio() {
    return this.bioForm.get('bio');
  }
}
```

```

get minLengthError() {
  const error = this.bio?.errors?.['minlength'];
  return error ? error : null;
}

get maxLengthError() {
  const error = this.bio?.errors?.['maxlength'];
  return error ? error : null;
}
}

```

Template:

```

<form [formGroup]="bioForm">
  <div>
    <label>Bio:</label>
    <textarea formControlName="bio" rows="4"></textarea>

    <p>Characters: {{ bio?.value?.length || 0 }} / 200</p>

    <div *ngIf="bio?.invalid && bio?.touched" style="color: red;">
      <div *ngIf="bio?.errors?.['required']">
        Bio is required
      </div>

      <div *ngIf="minLengthError">
        Bio must be at least {{ minLengthError.requiredLength }} characters.
        Current: {{ minLengthError.actualLength }} characters.
      </div>

      <div *ngIf="maxLengthError">
        Bio cannot exceed {{ maxLengthError.requiredLength }} characters.
        Current: {{ maxLengthError.actualLength }} characters.
      </div>
    </div>
  </div>
</form>

```

Output:

```
User types "Hello": - Characters: 5 / 200 - Error: "Bio must be at least 10
characters. Current: 5 characters." User types "Hello World!": - Characters: 12 / 200
- No error (valid) User types 210 characters: - Characters: 210 / 200 - Error: "Bio
cannot exceed 200 characters. Current: 210 characters."
```

3. Multiple Validators

3.1 Combining Multiple Validators

Component:

```
import { Component } from '@angular/core';
import { FormBuilder, FormGroup, Validators } from '@angular/forms';

@Component({
  selector: 'app-password-form',
  templateUrl: './password-form.component.html'
})
export class PasswordFormComponent {
  passwordForm: FormGroup;

  constructor(private fb: FormBuilder) {
    this.passwordForm = this.fb.group({
      password: ['', [
        Validators.required,
        Validators.minLength(8),
        Validators.maxLength(20),
        Validators.pattern('^(?=.*[a-z])(?=.*[A-Z])(?=.*[0-9]).*$')
        // Must contain: lowercase, uppercase, and number
      ]]
    });
  }

  get password() {
    return this.passwordForm.get('password');
  }
}
```

Template:

```
<form [formGroup]="passwordForm">
  <div>
    <label>Password:</label>
    <input type="password" formControlName="password">

    <div *ngIf="password?.invalid && password?.touched" style="color: red;">
      <div *ngIf="password?.errors?.['required']">
        ✗ Password is required
      </div>
      <div *ngIf="password?.errors?.['minlength']">
        ✗ Minimum 8 characters required
      </div>
      <div *ngIf="password?.errors?.['maxlength']">
        ✗ Maximum 20 characters allowed
      </div>
      <div *ngIf="password?.errors?.['pattern']">
        ✗ Must contain uppercase, lowercase, and number
      </div>
    </div>

    <div *ngIf="password?.valid" style="color: green;">
      ✓ Password is strong
    </div>
  </div>
</form>
```

Output:

User types "pass": - ✗ Minimum 8 characters required - ✗ Must contain uppercase, lowercase, and number
User types "password": - ✗ Must contain uppercase, lowercase, and number
User types "Password": - ✗ Must contain uppercase, lowercase, and number
User types "Password1": - ✓ Password is strong (all validations pass)

4. Custom Validators

4.1 Creating a Custom Validator Function

Validator File (custom-validators.ts):

```
import { AbstractControl, ValidationErrors } from '@angular/forms';

export class CustomValidators {
  // Validator: No spaces allowed
  static noSpaces(control: AbstractControl): ValidationErrors | null {
    if (control.value && control.value.indexOf(' ') !== -1) {
      return { noSpaces: true };
    }
    return null;
  }

  // Validator: Must be alphabetic only
  static alphabeticOnly(control: AbstractControl): ValidationErrors | null {
    const pattern = /^[a-zA-Z]+$/;
    if (control.value && !pattern.test(control.value)) {
      return { alphabeticOnly: true };
    }
    return null;
  }
}
```

Component:

```
import { Component } from '@angular/core';
import { FormBuilder, FormGroup, Validators } from '@angular/forms';
import { CustomValidators } from './custom-validators';

@Component({
  selector: 'app-user-form',
  templateUrl: './user-form.component.html'
})
export class UserFormComponent {
  userForm: FormGroup;

  constructor(private fb: FormBuilder) {
    this.userForm = this.fb.group({
      username: ['', [
        Validators.required,
        CustomValidators.noSpaces,
        Validators.minLength(3)
      ]],
      firstName: ['', [
        Validators.required,
        CustomValidators.alphabeticOnly
      ]],
      lastName: ['', [
        Validators.required,
        CustomValidators.alphabeticOnly
      ]],
      email: ['', [
        Validators.required,
        CustomValidators.noSpaces,
        CustomValidators.alphabeticOnly
      ]],
      password: ['', [
        Validators.required,
        CustomValidators.noSpaces,
        CustomValidators.alphabeticOnly,
        Validators.minLength(6)
      ]],
      confirmPassword: ['', [
        Validators.required,
        CustomValidators.noSpaces,
        CustomValidators.alphabeticOnly,
        Validators.minLength(6)
      ]],
    });
  }
}
```

```

    ]]
  });
}

get username() {
  return this.userForm.get('username');
}

get firstName() {
  return this.userForm.get('firstName');
}
}

```

Template:

```

<form [formGroup]="userForm">
  <div>
    <label>Username:</label>
    <input type="text" formControlName="username">

    <div *ngIf="username?.invalid && username?.touched" style="color: red;">
      <div *ngIf="username?.errors?.['required']">
        Username is required
      </div>
      <div *ngIf="username?.errors?.['noSpaces']">
        Username cannot contain spaces
      </div>
      <div *ngIf="username?.errors?.['minlength']">
        Username must be at least 3 characters
      </div>
    </div>
  </div>

  <div>
    <label>First Name:</label>
    <input type="text" formControlName="firstName">

    <div *ngIf="firstName?.invalid && firstName?.touched" style="color: red;">
      <div *ngIf="firstName?.errors?.['required']">
        First name is required
      </div>
      <div *ngIf="firstName?.errors?.['alphabeticOnly']">
        First name must contain only letters
      </div>
    </div>
  </div>

```

```
</div>
</div>
</form>
```

Output:

Username field: User types "john doe": - Error: "Username cannot contain spaces" User types "johndoe": - Valid (no spaces, at least 3 characters) First Name field: User types "John123": - Error: "First name must contain only letters" User types "John": - Valid (alphabetic only)

4.2 Parameterized Custom Validator

Validator File:

```
import { AbstractControl, ValidationErrors, ValidatorFn } from '@angular/forms';

export class CustomValidators {
  // Validator with parameter: forbid specific values
  static forbiddenValue(forbiddenValue: string): ValidatorFn {
    return (control: AbstractControl): ValidationErrors | null => {
      if (control.value && control.value.toLowerCase() === forbiddenValue.toLowerCase()) {
        return { forbiddenValue: { value: forbiddenValue } };
      }
      return null;
    };
  }

  // Validator: minimum age requirement
  static minimumAge(minAge: number): ValidatorFn {
    return (control: AbstractControl): ValidationErrors | null => {
      if (!control.value) {
        return null;
      }

      const birthDate = new Date(control.value);
      const today = new Date();
      let age = today.getFullYear() - birthDate.getFullYear();
      const monthDiff = today.getMonth() - birthDate.getMonth();

      if (monthDiff < 0 || (monthDiff === 0 && today.getDate() < birthDate.getDate())) {
        age--;
      }
    };
  }
}
```

```
    }

    if (age < minAge) {
      return { minimumAge: { requiredAge: minAge, actualAge: age } };
    }

    return null;
  };
}
}
```

Component:

```
import { Component } from '@angular/core';
import { FormBuilder, FormGroup, Validators } from '@angular/forms';
import { CustomValidators } from './custom-validators';

@Component({
  selector: 'app-registration-form',
  templateUrl: './registration-form.component.html'
})
export class RegistrationFormComponent {
  registrationForm: FormGroup;

  constructor(private fb: FormBuilder) {
    this.registrationForm = this.fb.group({
      username: ['', [
        Validators.required,
        CustomValidators.forbiddenValue('admin'),
        CustomValidators.forbiddenValue('root')
      ]],
      dateOfBirth: ['', [
        Validators.required,
        CustomValidators.minimumAge(18)
      ]]
    });
  }

  get username() {
    return this.registrationForm.get('username');
  }

  get dateOfBirth() {
```



```
    return this.registrationForm.get('dateOfBirth');  
  }  
}
```

Template:

```
<form [formGroup]="registrationForm">  
  <div>  
    <label>Username:</label>  
    <input type="text" formControlName="username">  
  
    <div *ngIf="username?.invalid && username?.touched" style="color: red;">  
      <div *ngIf="username?.errors?.['forbiddenValue']">  
        Username "{{ username.errors?.['forbiddenValue'].value }}" is not allowed  
      </div>  
    </div>  
  </div>  
  
  <div>  
    <label>Date of Birth:</label>  
    <input type="date" formControlName="dateOfBirth">  
  
    <div *ngIf="dateOfBirth?.invalid && dateOfBirth?.touched" style="color: red;">  
      <div *ngIf="dateOfBirth?.errors?.['minimumAge']">  
        You must be at least {{ dateOfBirth.errors?.['minimumAge'].requiredAge }} years old.  
        Your age: {{ dateOfBirth.errors?.['minimumAge'].actualAge }}  
      </div>  
    </div>  
  </div>  
</form>
```

Output:

Username field: User types "admin": - Error: 'Username "admin" is not allowed' User types "root": - Error: 'Username "root" is not allowed' User types "johndoe": - Valid
Date of Birth field: User selects "2010-05-15" (age 15): - Error: "You must be at least 18 years old. Your age: 15" User selects "2000-01-01" (age 26): - Valid

5. Cross-Field Validation

5.1 Comparing Two Fields

Validator File:

```
import { AbstractControl, ValidationErrors, ValidatorFn } from '@angular/forms';

export class CrossFieldValidators {
  // Validator: Two fields must match
  static matchFields(field1: string, field2: string): ValidatorFn {
    return (formGroup: AbstractControl): ValidationErrors | null => {
      const control1 = formGroup.get(field1);
      const control2 = formGroup.get(field2);

      if (!control1 || !control2) {
        return null;
      }

      if (control2.value && control1.value !== control2.value) {
        return { fieldsMismatch: true };
      }

      return null;
    };
  }
}
```

Component:

```
import { Component } from '@angular/core';
import { FormBuilder, FormGroup, Validators } from '@angular/forms';
import { CrossFieldValidators } from '../cross-field-validators';

@Component({
  selector: 'app-password-change',
  templateUrl: './password-change.component.html'
})
export class PasswordChangeComponent {
  passwordForm: FormGroup;

  constructor(private fb: FormBuilder) {
```

```

this.passwordForm = this.fb.group({
  password: ['', [Validators.required, Validators.minLength(6)]],
  confirmPassword: ['', Validators.required]
}, {
  validators: CrossFieldValidators.matchFields('password', 'confirmPassword')
});

get password() {
  return this.passwordForm.get('password');
}

get confirmPassword() {
  return this.passwordForm.get('confirmPassword');
}
}

```

Template:

```

<form [formGroup]="passwordForm">
  <div>
    <label>Password:</label>
    <input type="password" formControlName="password">

    <div *ngIf="password?.invalid && password?.touched" style="color: red;">
      <div *ngIf="password?.errors?.['required']">
        Password is required
      </div>
      <div *ngIf="password?.errors?.['minlength']">
        Password must be at least 6 characters
      </div>
    </div>
  </div>

  <div>
    <label>Confirm Password:</label>
    <input type="password" formControlName="confirmPassword">

    <div *ngIf="confirmPassword?.invalid && confirmPassword?.touched" style="color: red;">
      <div *ngIf="confirmPassword?.errors?.['required']">
        Please confirm your password
      </div>
    </div>

    <div *ngIf="passwordForm.errors?.['fieldsMismatch'] && confirmPassword?.touched"

```

```
        style="color: red;">
        Passwords do not match
    </div>
</div>

<button type="submit" [disabled]="passwordForm.invalid">
    Change Password
</button>
</form>
```

Output:

User types password: "secret123" User types confirm password: "secret12" - Error: "Passwords do not match" User corrects confirm password to: "secret123" - No error (passwords match) - Submit button enabled

5.2 Date Range Validation

Validator File:

```
import { AbstractControl, ValidationErrors, ValidatorFn } from '@angular/forms';

export class CrossFieldValidators {
    // Validator: End date must be after start date
    static dateRange(startDateField: string, endDateField: string): ValidatorFn {
        return (formGroup: AbstractControl): ValidationErrors | null => {
            const startDate = formGroup.get(startDateField);
            const endDate = formGroup.get(endDateField);

            if (!startDate || !endDate || !startDate.value || !endDate.value) {
                return null;
            }

            const start = new Date(startDate.value);
            const end = new Date(endDate.value);

            if (end <= start) {
                return { invalidDateRange: true };
            }

            return null;
        }
    }
}
```

```
    }  
  }  
}
```

Component:

```
import { Component } from '@angular/core';  
import { FormBuilder, FormGroup, Validators } from '@angular/forms';  
import { CrossFieldValidators } from './cross-field-validators';  
  
@Component({  
  selector: 'app-booking-form',  
  templateUrl: './booking-form.component.html'  
})  
export class BookingFormComponent {  
  bookingForm: FormGroup;  
  
  constructor(private fb: FormBuilder) {  
    this.bookingForm = this.fb.group({  
      checkIn: ['', Validators.required],  
      checkOut: ['', Validators.required]  
    }, {  
      validators: CrossFieldValidators.dateRange('checkIn', 'checkOut')  
    });  
  }  
  
  get checkIn() {  
    return this.bookingForm.get('checkIn');  
  }  
  
  get checkOut() {  
    return this.bookingForm.get('checkOut');  
  }  
}
```

Template:

```
<form [formGroup]="bookingForm">  
  <div>  
    <label>Check-in Date:</label>  
    <input type="date" formControlName="checkIn">  
  </div>
```

```
<div>
  <label>Check-out Date:</label>
  <input type="date" formControlName="checkOut">

  <div *ngIf="bookingForm.errors?.['invalidDateRange'] && checkOut?.touched"
    style="color: red;">
    Check-out date must be after check-in date
  </div>
</div>

<button type="submit" [disabled]="bookingForm.invalid">
  Book Now
</button>
</form>
```

Output:

User selects Check-in: 2026-03-15 User selects Check-out: 2026-03-10 - Error: "Check-out date must be after check-in date" User changes Check-out to: 2026-03-20 - No error (check-out is after check-in) - Submit button enabled

6. Dynamic Validation

6.1 Adding and Removing Validators

Component:

```
import { Component } from '@angular/core';
import { FormBuilder, FormGroup, Validators } from '@angular/forms';

@Component({
  selector: 'app-contact-form',
  templateUrl: './contact-form.component.html'
})
export class ContactFormComponent {
  contactForm: FormGroup;

  constructor(private fb: FormBuilder) {
    this.contactForm = this.fb.group({
```

```

    contactMethod: ['email'],
    email: ['', [Validators.required, Validators.email]],
    phone: ['']
  });

  // Listen to contact method changes
  this.contactForm.get('contactMethod')?.valueChanges.subscribe(method => {
    this.updateValidation(method);
  });
}

updateValidation(method: string) {
  const emailControl = this.contactForm.get('email');
  const phoneControl = this.contactForm.get('phone');

  if (method === 'email') {
    emailControl?.setValidators([Validators.required, Validators.email]);
    phoneControl?.clearValidators();
  } else {
    phoneControl?.setValidators([Validators.required, Validators.pattern('^[0-9]{10}$')]);
    emailControl?.clearValidators();
  }

  emailControl?.updateValueAndValidity();
  phoneControl?.updateValueAndValidity();
}

get email() {
  return this.contactForm.get('email');
}

get phone() {
  return this.contactForm.get('phone');
}
}

```

Template:

```

<form [formGroup]="contactForm">
  <div>
    <label>Preferred Contact Method:</label>
    <select formControlName="contactMethod">
      <option value="email">Email</option>
      <option value="phone">Phone</option>
    </select>
  </div>

```

```

</div>

<div *ngIf="contactForm.value.contactMethod === 'email'">
  <label>Email:</label>
  <input type="email" formControlName="email">

  <div *ngIf="email?.invalid && email?.touched" style="color: red;">
    <div *ngIf="email?.errors?.['required']">Email is required</div>
    <div *ngIf="email?.errors?.['email']">Invalid email format</div>
  </div>
</div>

<div *ngIf="contactForm.value.contactMethod === 'phone'">
  <label>Phone:</label>
  <input type="tel" formControlName="phone">

  <div *ngIf="phone?.invalid && phone?.touched" style="color: red;">
    <div *ngIf="phone?.errors?.['required']">Phone is required</div>
    <div *ngIf="phone?.errors?.['pattern']">Phone must be 10 digits</div>
  </div>
</div>

<button type="submit" [disabled]="contactForm.invalid">Submit</button>
</form>

```

Output:

Initial state (Email selected): - Email field visible and required - Phone field hidden and not required
 User selects "Phone": - Email field hidden and no longer required - Phone field appears and becomes required - Phone validation: must be 10 digits
 User types "12345": - Error: "Phone must be 10 digits"
 User types "1234567890": - Valid (10 digits)

6.2 Conditional Validation Based on Checkbox

Component:

```

import { Component } from '@angular/core';
import { FormBuilder, FormGroup, Validators } from '@angular/forms';

@Component({
  selector: 'app-shipping-form',

```



```
templateUrl: './shipping-form.component.html'
})
export class ShippingFormComponent {
  shippingForm: FormGroup;

  constructor(private fb: FormBuilder) {
    this.shippingForm = this.fb.group({
      differentAddress: [false],
      shippingStreet: [''],
      shippingCity: [''],
      shippingZip: ['']
    });

    this.shippingForm.get('differentAddress')?.valueChanges.subscribe(checked => {
      this.updateShippingValidation(checked);
    });
  }

  updateShippingValidation(required: boolean) {
    const street = this.shippingForm.get('shippingStreet');
    const city = this.shippingForm.get('shippingCity');
    const zip = this.shippingForm.get('shippingZip');

    if (required) {
      street?.setValidators(Validators.required);
      city?.setValidators(Validators.required);
      zip?.setValidators([Validators.required, Validators.pattern('^[0-9]{5}$')]);
    } else {
      street?.clearValidators();
      city?.clearValidators();
      zip?.clearValidators();
    }

    street?.updateValueAndValidity();
    city?.updateValueAndValidity();
    zip?.updateValueAndValidity();
  }
}
```

Template:

```
<form [formGroup]="shippingForm">
  <div>
    <label>
      <input type="checkbox" formControlName="differentAddress">
```

```
        Use different shipping address
    </label>
</div>

<div *ngIf="shippingForm.value.differentAddress">
    <h4>Shipping Address</h4>

    <div>
        <label>Street:</label>
        <input type="text" formControlName="shippingStreet">
    </div>

    <div>
        <label>City:</label>
        <input type="text" formControlName="shippingCity">
    </div>

    <div>
        <label>Zip Code:</label>
        <input type="text" formControlName="shippingZip">
    </div>
</div>

<button type="submit" [disabled]="shippingForm.invalid">Continue</button>
</form>
```

Output:

Initial state (checkbox unchecked): - Shipping address fields hidden - Form is valid - Submit button enabled
User checks "Use different shipping address": - Shipping address fields appear - All shipping fields become required - Form becomes invalid (fields empty) - Submit button disabled
User fills shipping address: - Form becomes valid - Submit button enabled
User unchecks the checkbox: - Shipping fields hidden - Validators removed - Form valid again

7. Practical Example: User Registration with Validation

Let's build a complete registration form with all validation techniques.

Validators (app-validators.ts):

```
import { AbstractControl, ValidationErrors, ValidatorFn } from '@angular/forms';

export class AppValidators {
  // No whitespace allowed
  static noWhiteSpace(control: AbstractControl): ValidationErrors | null {
    const isWhitespace = (control.value || '').trim().length === 0;
    return isWhitespace && control.value ? { whitespace: true } : null;
  }

  // Username format validator
  static usernameFormat(control: AbstractControl): ValidationErrors | null {
    const pattern = /^[a-zA-Z0-9_]+$ /;
    if (control.value && !pattern.test(control.value)) {
      return { usernameFormat: true };
    }
    return null;
  }

  // Strong password validator
  static strongPassword(control: AbstractControl): ValidationErrors | null {
    const value = control.value || '';
    const hasUpperCase = /[A-Z]/.test(value);
    const hasLowerCase = /[a-z]/.test(value);
    const hasNumber = /[0-9]/.test(value);
    const hasSpecialChar = /[!@#$$%^&*(),.?":{}|<>]/.test(value);

    const errors: any = {};

    if (!hasUpperCase) errors.noUpperCase = true;
    if (!hasLowerCase) errors.noLowerCase = true;
    if (!hasNumber) errors.noNumber = true;
    if (!hasSpecialChar) errors.noSpecialChar = true;

    return Object.keys(errors).length > 0 ? errors : null;
  }

  // Password match validator (for FormGroup)
  static passwordsMatch(control: AbstractControl): ValidationErrors | null {
    const password = control.get('password');
    const confirmPassword = control.get('confirmPassword');

    if (!password || !confirmPassword) {
      return null;
    }

    return password.value === confirmPassword.value ? null : { passwordsMismatch: true };
  }
}
```

```

}

// Age validator
static minimumAge(minAge: number): ValidatorFn {
  return (control: AbstractControl): ValidationErrors | null => {
    if (!control.value) return null;

    const birthDate = new Date(control.value);
    const today = new Date();
    let age = today.getFullYear() - birthDate.getFullYear();
    const monthDiff = today.getMonth() - birthDate.getMonth();

    if (monthDiff < 0 || (monthDiff === 0 && today.getDate() < birthDate.getDate())) {
      age--;
    }

    return age < minAge ? { minimumAge: { required: minAge, actual: age } } : null;
  };
}

```

Service (user.service.ts):

```

import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class UserService {
  countries = ['USA', 'India', 'UK', 'Canada', 'Australia'];

  statesByCountry: any = {
    'USA': ['California', 'Texas', 'New York', 'Florida'],
    'India': ['Maharashtra', 'Karnataka', 'Delhi', 'Tamil Nadu'],
    'UK': ['England', 'Scotland', 'Wales', 'Northern Ireland'],
    'Canada': ['Ontario', 'Quebec', 'British Columbia', 'Alberta'],
    'Australia': ['New South Wales', 'Victoria', 'Queensland', 'Western Australia']
  };

  getStates(country: string): string[] {
    return this.statesByCountry[country] || [];
  }

  registerUser(userData: any): boolean {
    console.log('Registering user:', userData);
  }
}

```

```
    return true;
  }
}
```

Component (registration.component.ts):

```
import { Component } from '@angular/core';
import { FormBuilder, FormGroup, Validators } from '@angular/forms';
import { AppValidators } from './app-validators';
import { UserService } from './user.service';
```

```
@Component({
  selector: 'app-registration',
  templateUrl: './registration.component.html',
  styles: [`
    .form-container {
      max-width: 600px;
      margin: 20px auto;
      padding: 20px;
      border: 1px solid #ddd;
      border-radius: 8px;
    }
    .form-section {
      margin: 20px 0;
      padding: 15px;
      background: #f9f9f9;
      border-radius: 5px;
    }
    .form-section h3 {
      margin-top: 0;
      color: #333;
    }
    .form-group {
      margin: 15px 0;
    }
    label {
      display: block;
      font-weight: bold;
      margin-bottom: 5px;
    }
    input, select {
      width: 100%;
      padding: 8px;
      border: 1px solid #ddd;
      border-radius: 4px;
    }
  `]
})
```

```
    box-sizing: border-box;
}
input.ng-invalid.ng-touched,
select.ng-invalid.ng-touched {
    border-color: #dc3545;
}
input.ng-valid.ng-touched,
select.ng-valid.ng-touched {
    border-color: #28a745;
}
.error {
    color: #dc3545;
    font-size: 12px;
    margin-top: 5px;
}
.success {
    color: #28a745;
    font-size: 12px;
    margin-top: 5px;
}
.password-requirements {
    font-size: 12px;
    margin-top: 5px;
}
.requirement {
    margin: 2px 0;
}
.requirement.valid {
    color: #28a745;
}
.requirement.invalid {
    color: #dc3545;
}
button {
    padding: 12px 30px;
    margin: 10px 5px 0 0;
    border: none;
    border-radius: 4px;
    cursor: pointer;
    font-size: 14px;
}
.btn-primary {
    background: #007bff;
    color: white;
}
.btn-primary.disabled {
    background: #ccc;
```

```

    cursor: not-allowed;
  }
  .btn-secondary {
    background: #6c757d;
    color: white;
  }
  .success-message {
    background: #d4edda;
    color: #155724;
    padding: 20px;
    border-radius: 5px;
    text-align: center;
  }
  `]
})
export class RegistrationComponent {
  registrationForm: FormGroup;
  submitted = false;
  availableStates: string[] = [];

  constructor(
    private fb: FormBuilder,
    public userService: UserService
  ) {
    this.registrationForm = this.fb.group({
      // Account Information
      account: this.fb.group({
        username: ['', [
          Validators.required,
          Validators.minLength(4),
          Validators.maxLength(20),
          AppValidators.noWhitespace,
          AppValidators.usernameFormat
        ]],
        email: ['', [
          Validators.required,
          Validators.email
        ]],
        password: ['', [
          Validators.required,
          Validators.minLength(8),
          AppValidators.strongPassword
        ]],
        confirmPassword: ['', Validators.required]
      }, { validators: AppValidators.passwordsMatch }),

      // Personal Information

```

```
personal: this.fb.group({
  firstName: ['', [
    Validators.required,
    Validators.minLength(2),
    AppValidators.noWhitespace
  ]],
  lastName: ['', [
    Validators.required,
    Validators.minLength(2),
    AppValidators.noWhitespace
  ]],
  dateOfBirth: ['', [
    Validators.required,
    AppValidators.minimumAge(18)
  ]],
  gender: ['', Validators.required]
}),

// Address Information
address: this.fb.group({
  country: ['', Validators.required],
  state: ['', Validators.required],
  city: ['', Validators.required],
  zipCode: ['', [
    Validators.required,
    Validators.pattern('^[0-9]{5,6}$')
  ]],
}),

// Terms
acceptTerms: [false, Validators.requiredTrue]
});

// Listen to country changes
this.address.get('country')?.valueChanges.subscribe(country => {
  this.onCountryChange(country);
});
}

get account() {
  return this.registrationForm.get('account') as FormGroup;
}

get personal() {
  return this.registrationForm.get('personal') as FormGroup;
}
```



```
get address() {
  return this.registrationForm.get('address') as FormGroup;
}

get username() {
  return this.account.get('username');
}

get email() {
  return this.account.get('email');
}

get password() {
  return this.account.get('password');
}

get confirmPassword() {
  return this.account.get('confirmPassword');
}

get dateOfBirth() {
  return this.personal.get('dateOfBirth');
}

onCountryChange(country: string) {
  this.availableStates = this.userService.getStates(country);
  this.address.patchValue({ state: '' });
}

hasPasswordError(errorType: string): boolean {
  const errors = this.password?.errors;
  return errors && errors[errorType];
}

onSubmit() {
  if (this.registrationForm.valid) {
    const success = this.userService.registerUser(this.registrationForm.value);
    if (success) {
      this.submitted = true;
      console.log('Registration successful:', this.registrationForm.value);
    }
  } else {
    this.markFormGroupTouched(this.registrationForm);
  }
}

markFormGroupTouched(formGroup: FormGroup) {
```

```

Object.keys(formGroup.controls).forEach(key => {
  const control = formGroup.get(key);
  control?.markAsTouched();

  if (control instanceof FormGroup) {
    this.markFormGroupTouched(control);
  }
});
}

onReset() {
  this.registrationForm.reset();
  this.submitted = false;
}
}

```

Template (registration.component.html):

```

<div class="form-container">
  <h2>User Registration</h2>

  <!-- Success Message -->
  <div *ngIf="submitted" class="success-message">
    <h3>✓ Registration Successful!</h3>
    <p>Welcome, {{ account.value.username }}!</p>
    <p>Your account has been created successfully.</p>
    <button class="btn-primary" (click)="onReset()">Register Another User</button>
  </div>

  <!-- Registration Form -->
  <form [formGroup]="registrationForm" (ngSubmit)="onSubmit()" *ngIf="!submitted">

    <!-- Account Information -->
    <div class="form-section" formGroupName="account">
      <h3>Account Information</h3>

      <div class="form-group">
        <label>Username</label>
        <input type="text" formControlName="username">
        <div *ngIf="username?.invalid && username?.touched" class="error">
          <div *ngIf="username?.errors?.['required']">Username is required</div>
          <div *ngIf="username?.errors?.['minlength']">Minimum 4 characters required</div>
          <div *ngIf="username?.errors?.['maxlength']">Maximum 20 characters allowed</div>
          <div *ngIf="username?.errors?.['whitespace']">Username cannot be only whitespace</div>
        </div>
      </div>
    </div>
  </form>

```

```

    <div *ngIf="username?.errors?.['usernameFormat']">Only letters, numbers, and
underscores allowed</div>
  </div>
</div>

<div class="form-group">
  <label>Email</label>
  <input type="email" formControlName="email">
  <div *ngIf="email?.invalid && email?.touched" class="error">
    <div *ngIf="email?.errors?.['required']">Email is required</div>
    <div *ngIf="email?.errors?.['email']">Invalid email format</div>
  </div>
</div>

<div class="form-group">
  <label>Password</label>
  <input type="password" formControlName="password">

  <div class="password-requirements" *ngIf="password?.touched">
    <div class="requirement" [class.valid]="!hasPasswordError('noUpperCase')">
      [class.invalid]="hasPasswordError('noUpperCase')">
        {{ !hasPasswordError('noUpperCase') ? '✓' : '✗' }} At least one uppercase letter
    </div>
    <div class="requirement" [class.valid]="!hasPasswordError('noLowerCase')">
      [class.invalid]="hasPasswordError('noLowerCase')">
        {{ !hasPasswordError('noLowerCase') ? '✓' : '✗' }} At least one lowercase letter
    </div>
    <div class="requirement" [class.valid]="!hasPasswordError('noNumber')">
      [class.invalid]="hasPasswordError('noNumber')">
        {{ !hasPasswordError('noNumber') ? '✓' : '✗' }} At least one number
    </div>
    <div class="requirement" [class.valid]="!hasPasswordError('noSpecialChar')">
      [class.invalid]="hasPasswordError('noSpecialChar')">
        {{ !hasPasswordError('noSpecialChar') ? '✓' : '✗' }} At least one special
character
    </div>
    <div class="requirement" [class.valid]="!password?.errors?.['minlength']">
      [class.invalid]="password?.errors?.['minlength']">
        {{ !password?.errors?.['minlength'] ? '✓' : '✗' }} Minimum 8 characters
    </div>
  </div>
</div>

<div class="form-group">
  <label>Confirm Password</label>
  <input type="password" formControlName="confirmPassword">
  <div *ngIf="confirmPassword?.touched" class="error">

```

```

    <div *ngIf="confirmPassword?.errors?.['required']">Please confirm your password</div>
    <div *ngIf="account.errors?.['passwordsMismatch'] && !confirmPassword?.errors?.
['required']">
      Passwords do not match
    </div>
  </div>
</div>

<!-- Personal Information -->
<div class="form-section" formGroupName="personal">
  <h3>Personal Information</h3>

  <div class="form-group">
    <label>First Name</label>
    <input type="text" formControlName="firstName">
    <div *ngIf="personal.get('firstName')?.invalid && personal.get('firstName')?.touched"
class="error">
      <div *ngIf="personal.get('firstName')?.errors?.['required']">First name is required</
div>
      <div *ngIf="personal.get('firstName')?.errors?.['minlength']">Minimum 2 characters</
div>
    </div>
  </div>

  <div class="form-group">
    <label>Last Name</label>
    <input type="text" formControlName="lastName">
    <div *ngIf="personal.get('lastName')?.invalid && personal.get('lastName')?.touched"
class="error">
      <div *ngIf="personal.get('lastName')?.errors?.['required']">Last name is required</
div>
      <div *ngIf="personal.get('lastName')?.errors?.['minlength']">Minimum 2 characters</
div>
    </div>
  </div>

  <div class="form-group">
    <label>Date of Birth</label>
    <input type="date" formControlName="dateOfBirth">
    <div *ngIf="dateOfBirth?.invalid && dateOfBirth?.touched" class="error">
      <div *ngIf="dateOfBirth?.errors?.['required']">Date of birth is required</div>
      <div *ngIf="dateOfBirth?.errors?.['minimumAge']">
        You must be at least {{ dateOfBirth.errors?.['minimumAge'].required }} years old
        (Current age: {{ dateOfBirth.errors?.['minimumAge'].actual }})
      </div>
    </div>
  </div>

```

```

</div>

<div class="form-group">
  <label>Gender</label>
  <select formControlName="gender">
    <option value="">Select Gender</option>
    <option value="male">Male</option>
    <option value="female">Female</option>
    <option value="other">Other</option>
  </select>
</div>
</div>

<!-- Address Information -->
<div class="form-section" formGroupName="address">
  <h3>Address</h3>

  <div class="form-group">
    <label>Country</label>
    <select formControlName="country">
      <option value="">Select Country</option>
      <option *ngFor="let country of userService.countries" [value]="country">
        {{ country }}
      </option>
    </select>
  </div>

  <div class="form-group" *ngIf="address.value.country">
    <label>State</label>
    <select formControlName="state">
      <option value="">Select State</option>
      <option *ngFor="let state of availableStates" [value]="state">
        {{ state }}
      </option>
    </select>
  </div>

  <div class="form-group">
    <label>City</label>
    <input type="text" formControlName="city">
  </div>

  <div class="form-group">
    <label>Zip Code</label>
    <input type="text" formControlName="zipCode">
    <div *ngIf="address.get('zipCode')?.invalid && address.get('zipCode')?.touched"
class="error">

```

```

      <div *ngIf="address.get('zipCode')?.errors?.['pattern']">Zip code must be 5-6
digits</div>
    </div>
  </div>
</div>

<!-- Terms and Conditions -->
<div class="form-group">
  <label>
    <input type="checkbox" formControlName="acceptTerms">
    I accept the terms and conditions
  </label>
  <div *ngIf="registrationForm.get('acceptTerms')?.invalid &&
    registrationForm.get('acceptTerms')?.touched" class="error">
    You must accept the terms and conditions
  </div>
</div>

<!-- Form Actions -->
<div>
  <button type="submit" class="btn-primary" [disabled]="registrationForm.invalid">
    Register
  </button>
  <button type="button" class="btn-secondary" (click)="onReset()">
    Reset
  </button>
</div>

<!-- Form Status -->
<div style="margin-top: 20px; padding: 10px; background: #f5f5f5; font-size: 12px;">
  Form Status: {{ registrationForm.valid ? 'Valid ✓' : 'Invalid ✗' }}
</div>
</form>
</div>

```

Output:

User fills the registration form: Account Information: - Username: "john_doe123" ✓ - Email: "john@example.com" ✓ - Password: "SecurePass123!" ✓ At least one uppercase letter ✓ At least one lowercase letter ✓ At least one number ✓ At least one special character ✓ Minimum 8 characters - Confirm Password: "SecurePass123!" ✓ (matches) Personal Information: - First Name: "John" ✓ - Last Name: "Doe" ✓ - Date of Birth: "1995-05-15" ✓ (Age: 30, meets minimum 18) - Gender: "Male" ✓ Address: - Country: "USA" → State dropdown populates - State: "California" ✓ - City: "San Francisco" ✓ -

```
Zip Code: "94102" ✓ (5 digits) Terms: ✓ Checked Form Status: Valid ✓ Register button
enabled Clicks Register: Console: Registration successful: { account: { username:
"john_doe123", email: "john@example.com", ... }, personal: { firstName: "John",
lastName: "Doe", ... }, address: { country: "USA", state: "California", ... },
acceptTerms: true } Success message displays
```

8. Practical Task

Task: Build a Loan Application Form with Complete Validation

Requirements:

1. Create a Loan Application Form with validation:

Applicant Information (FormGroup):

- Full Name (required, min 3 characters, no whitespace only)
- Email (required, valid email format)
- Phone (required, exactly 10 digits)
- PAN Number (required, pattern: 5 letters + 4 digits + 1 letter, e.g., ABCDE1234F)
- Date of Birth (required, minimum age 21)
- Employment Status (dropdown: Employed, Self-Employed, Unemployed) (required)

Employment Details (FormGroup - conditional):

- Show only if Employment Status is "Employed" or "Self-Employed"
- Company Name (required if shown)
- Monthly Income (required if shown, minimum 15000)
- Years of Experience (required if shown, min 0, max 50)

Loan Details (FormGroup):

- Loan Type (dropdown: Personal, Home, Vehicle, Education) (required)
- Loan Amount (required, min 50000, max 10000000)

- Loan Tenure (months) (required, min 6, max 360)
- Purpose (textarea, required, min 20 characters, max 500)

Co-Applicant (optional checkbox with conditional fields):

- Has Co-Applicant (checkbox)
- If checked, show:
 - Co-Applicant Name (required)
 - Co-Applicant Email (required, valid email)
 - Relationship (dropdown: Spouse, Parent, Sibling, Friend) (required)

2. Custom Validators to Create:

- `panNumberValidator` : Format AAAAAA9999A
- `minimumIncome(amount)` : Parameterized validator for minimum income
- `loanAmountValidator` : For Personal loan, max 500000; for Home loan, no restriction
- Cross-field: If Employment Status is "Unemployed", disable employment fields and loan application

3. Dynamic Validation:

- Show/hide employment details based on employment status
- Add/remove validators for co-applicant fields based on checkbox
- Disable form submission if employment status is "Unemployed"

4. Display Requirements:

- Show all validation errors clearly
- Calculate and display:
 - Estimated EMI (Loan Amount / Tenure in months)
 - Eligibility Status based on income (if monthly income > 3 × EMI, show "Likely Eligible")
- Success message after submission with loan details

Expected Console Output:


```

Loan Application: {
  applicant: {
    fullName: "Rajesh Kumar",
    email: "rajesh@example.com",
    phone: "9876543210",
    panNumber: "ABCDE1234F",
    dateOfBirth: "1990-05-15",
    employmentStatus: "Employed"
  },
  employment: {
    companyName: "Tech Solutions Ltd",
    monthlyIncome: 75000,
    yearsOfExperience: 5
  },
  loan: {
    loanType: "Home",
    loanAmount: 2500000,
    loanTenure: 240,
    purpose: "Purchasing a 2BHK apartment in city center for family"
  },
  coApplicant: {
    hasCoApplicant: true,
    name: "Priya Kumar",
    email: "priya@example.com",
    relationship: "Spouse"
  }
}
Estimated EMI: 10417
Eligibility: Likely Eligible (Income: 75000, Required: 31251)

```

9. Summary

Key Concepts Covered

Concept	Description	Use Case
Built-in Validators	required, min, max, minLength, maxLength, pattern, email	Common validation scenarios
Error Messages		User feedback

Concept	Description	Use Case
	Displaying specific error messages for each validator	
Custom Validators	Creating reusable validation functions	Business-specific rules
Cross-Field Validation	Validators that compare multiple fields	Password match, date ranges
Dynamic Validation	Adding/removing validators at runtime	Conditional requirements

Built-in Validators Reference

Validator	Syntax	Description
required	<code>Validators.required</code>	Field cannot be empty
minLength	<code>Validators.minLength(n)</code>	Minimum string length
maxLength	<code>Validators.maxLength(n)</code>	Maximum string length
min	<code>Validators.min(n)</code>	Minimum numeric value
max	<code>Validators.max(n)</code>	Maximum numeric value
pattern	<code>Validators.pattern(regex)</code>	Must match regex pattern
email	<code>Validators.email</code>	Valid email format
requiredTrue	<code>Validators.requiredTrue</code>	Must be true (checkboxes)

Best Practices

1. Validator Organization:

- Create a separate file for custom validators
- Use static methods in a class
- Group related validators together

2. Error Messages:

- Show errors only when field is touched
- Use specific, helpful error messages
- Show one error at a time (priority: required → format → range)

3. Dynamic Validation:

- Call `updateValueAndValidity()` after changing validators
- Clear validators properly with `clearValidators()`
- Use `valueChanges` to listen for field changes

4. Cross-Field Validation:

- Apply at FormGroup level, not FormControl
- Check for null controls before validation
- Return null for valid, error object for invalid

5. Performance:

- Use getter methods for frequently accessed controls
- Avoid complex validation in templates
- Use `updateOn: 'blur'` for expensive validations

Common Patterns

Custom Validator Pattern:

```
static validatorName(control: AbstractControl): ValidationErrors | null {  
  if (/* invalid condition */) {  
    return { errorKey: true };  
  }  
  return null;  
}
```

Parameterized Validator Pattern:

```
static validatorName(param: any): ValidatorFn {  
  return (control: AbstractControl): ValidationErrors | null => {  
    // validation logic using param  
  }  
}
```

```
    return isValid ? null : { errorKey: { param } };  
  };  
}
```

Cross-Field Validator Pattern:

```
static crossFieldValidator(formGroup: AbstractControl): ValidationErrors | null {  
  const field1 = formGroup.get('field1');  
  const field2 = formGroup.get('field2');  
  // compare fields  
  return valid ? null : { errorKey: true };  
}
```

Dynamic Validation Pattern:

```
control.setValidators([Validators.required, ...]);  
control.clearValidators();  
control.updateValueAndValidity();
```

What's Next?

In **Module 10.3: Reactive Forms - Form Handling**, you'll learn:

- ValueChanges and StatusChanges observables
- Patching and setting form values
- Resetting forms
- Submitting forms with proper handling
- Form state management
- Best practices for form handling

Module Complete! You now have comprehensive knowledge of reactive forms validation, including built-in validators, custom validators, cross-field validation, and dynamic validation. You can build production-ready forms with robust validation logic.